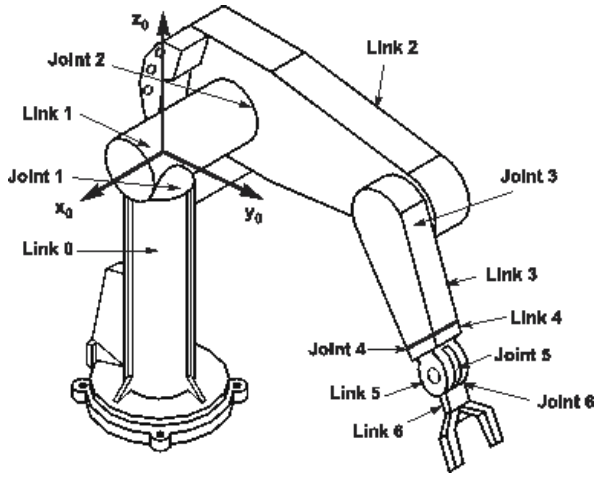
Lecture 11: Applications of the Jacobian Matrix

Iñigo Iturrate
Associate Professor
SDU Robotics,
The Maersk McKinney Moller Institute,
University of Southern Denmark

Ø27-604-3
inju@mmmi.sdu.dk



SDU ROBOTICS

Recap (Exercise, 10 min.)

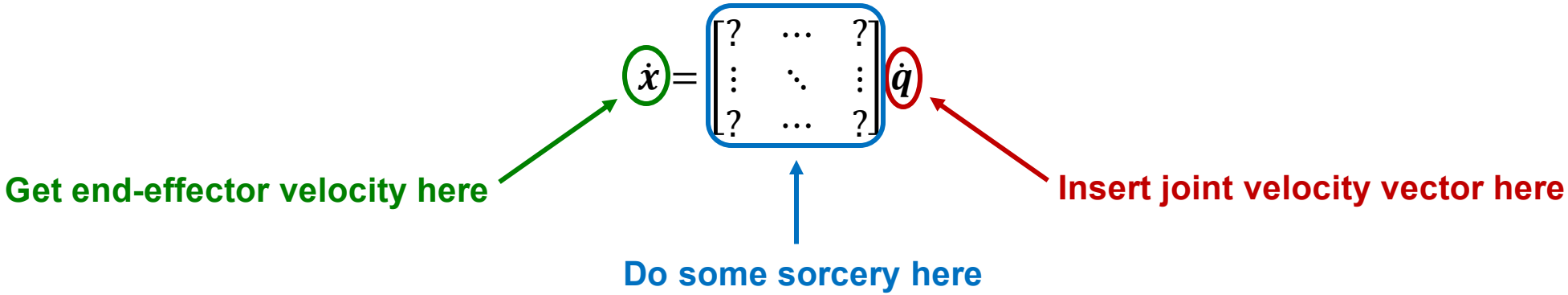
Discuss with your neighbors (5 min.):

In the context of robotics:
1. Why do we use the Jacobian?
2. How is the matrix structured?
3. What can we say about its individual elements in relation to how the robot moves in joint/Cartesian space?

Then share with me what you have discussed (5 min.).

What did we derive last time?

We wanted to find:



(This sorcery is called the Jacobian Matrix)

$$\dot{x} = J(q)\dot{q}$$

The Two Jacobians in Robotics

Remember that we can **describe Cartesian-space velocity** in (at least) **two ways**, depending on how we describe **rotational velocity**:

- Using **angular** velocity: $\dot{x} = \begin{bmatrix} \dot{p} \\ \omega \end{bmatrix}$
- Using the **derivative of Euler angles**: $\dot{x} = v = \begin{bmatrix} \dot{p} \\ \dot{\phi} \end{bmatrix}$

We can split the Jacobian in two: $J = \begin{bmatrix} J_P \\ J_O \end{bmatrix}$

Jacobian for Position *(This stays the same)*

Jacobian for Orientation

J_O based on angular velocity

J_O based on derivative of Euler angles

Geometric Jacobian

We will only study this

Analytical Jacobian

Building a Geometric Jacobian

We first **split the matrix** into (3×1) column vectors, $\boldsymbol{J}_{Pi}$ and $\boldsymbol{J}_{Oi}$, where each element i represents the **contribution of a single joint q_i to either the position or orientation**:

$$\boldsymbol{J} = \begin{bmatrix} \boldsymbol{J}_{P1} & \cdots & \boldsymbol{J}_{Pn} \\ \boldsymbol{J}_{O1} & & \boldsymbol{J}_{On} \end{bmatrix} \qquad \boldsymbol{J}_{Pi} = \begin{bmatrix} J_{Pi,x} \\ J_{Pi,y} \\ J_{Pi,z} \end{bmatrix} \qquad \boldsymbol{J}_{Oi} = \begin{bmatrix} J_{Oi,\omega_x} \\ J_{Oi,\omega_y} \\ J_{Oi,\omega_z} \end{bmatrix}$$

Then, for each joint i:

- **Revolute**: $\boldsymbol{J}_{Pi} = {}^0\hat{Z}_i \times ({}^0P_e - {}^0P_i)$, $\boldsymbol{J}_{Oi} = {}^0\hat{Z}_i$
- **Prismatic:** $\boldsymbol{J}_{Pi} = {}^0\hat{Z}_i$, $\boldsymbol{J}_{Oi} = 0$

$${}^0\hat{Z}_i = {}^0_iR \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

${}^0P_e$ is the position of the end-effector

${}^0P_i$ is the position of the origin of frame {i}

Singularities: Non-existence of inverse

When does the inverse of a matrix not exist?

Due to a **non-square** matrix:

- Example: **a redundant robot** (e.g. 7-DOF in a 3D space)

$$\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$$

$$\begin{bmatrix} \dot{\boldsymbol{p}} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \boldsymbol{J}_{P1} & \cdots & \boldsymbol{J}_{P7} \\ \boldsymbol{J}_{O1} & & \boldsymbol{J}_{O7} \end{bmatrix} \begin{bmatrix} q_1 \\ \vdots \\ q_7 \end{bmatrix}$$

$(6 \times 1) \qquad (6 \times 7) \qquad (7 \times 1)$



Due to a **rank-defficient square** matrix

What are we doing today?

1. Introduction to Robotics & Recap of Linear Algebra and Mathematical Notation
2. Translations & Rotation Matrices
3. Other Representations for Orientation
4. Transformation Matrices
5. Forward Kinematics & DH Parameters
6. Inverse Kinematics

7. Kinematic Simulation
8. Schematic Representations of Robots, Modeling a Real Robot

9. Angular Velocities, Velocity Propagation & Trajectory Generation
10. The Jacobian Matrix, Singularities
11. **Applications of the Jacobian Matrix** ***(Today)***
12. Lab Demonstration, Course Recap, Work on Physical Robot



"Robots that stay still",
i.e. positions/orientations

Simulation

"Robots that move",
i.e. velocities

SDU ROBOTICS

Topics for Today

Part I: Numerical Inverse Kinematics

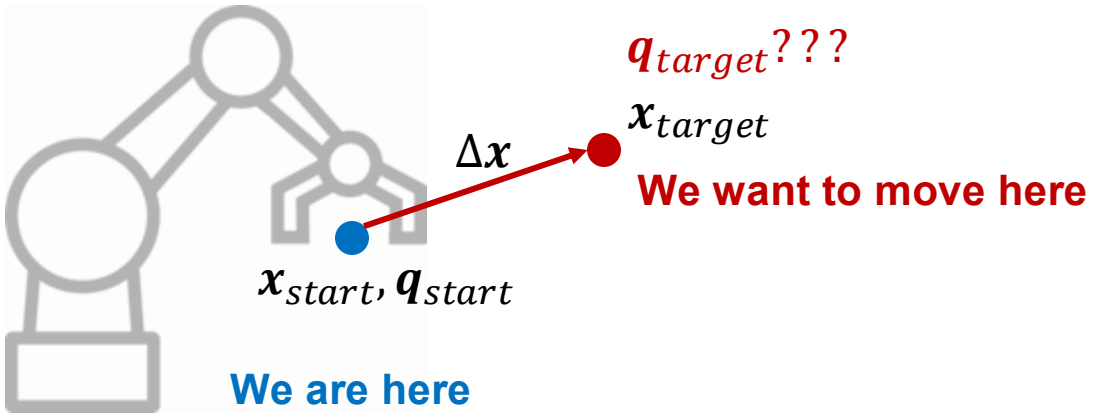
- Numerical IK Conceptually
- The Newton-Raphson Algorithm

Part II: Manipulability

- Manipulability Measures
- Eigen-decomposition
- Manipulability Ellipsoid
- Practical Applications

Part I: Numerical IK

Numerical Inverse Kinematics: Conceptually



1. We know: $x_{start}, q_{start} x_{target}$
2. We want to find: q_{target}
3. We can calculate: $\Delta x = x_{target} - x_{start}$
4. We take Δx and split it in smaller segments:



5. We make a very small step in the direction of Δx:

$$dq(t) = J^{-1}(q)dx(t)$$
$$q(t + dt) = q(t) + dq(t)dt$$

6. We re-calculate: $J\big(q(t + dt)\big), x(t + dt)$ *(from FK)*
7. We check if we are close enough to our target:
   - If yes: stop
   - If not: repeat 3-7.

What happens if, in step 5, the step size is not small enough?

Formally: Numerical IK using Newton-Raphson

Let us **reformulate** the IK as a **nonlinear optimization problem.** We have:

- A desired Cartesian space pose, x_d
- The desired IK solution, a set of joint angles, θ_d
- A **nonlinear** vector function that computes the FK, $f(\theta_d)$, where $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$

We want: $x_d - f(\theta_d) = 0$ (*this is our **objective function***)

We first apply Taylor expansion:

$$x_d = f(\theta_d) = f(\theta_0) + \underbrace{\left.\frac{\partial f}{\partial x}\right|_{\theta_0}}_{J(\theta_0)} \underbrace{(\theta_d - \theta_0)}_{\Delta\theta} + \cancel{\text{higher order terms}}$$

By ignoring these, we are **linearizing** around θ_0

$$J(\theta_0)\Delta\theta = x_d - f(\theta_0) \longrightarrow \Delta\theta = J^{-1}(\theta_0)\big(x_d - f(\theta_0)\big)$$

If we use this to take a **new step**: $\theta_1 = \theta_0 + \Delta\theta$, then this is **guaranteed to be closer to the solution**.

We can therefore **apply this recursively** until we are ”close enough”, i.e. $x_d - f(\theta_d) < \epsilon$.

IK: Newton Raphson Algorithm

Algorithm 1 Newton-Rhapson method for numerical IK

Require: θ, $x^* = f(\theta)$, x_d and $x_d \approx x^*$

Compute $\Delta x = x_d - x^*$

while $\|\Delta x\| > \epsilon$ **do**

Compute $J(\theta)$

Solve $J(\theta)\Delta\theta = \Delta x$ for $\Delta\theta$

$\theta := \theta + \Delta\theta$

Compute $x^* = f(\theta)$

Compute $\Delta x = x_d - x^*$

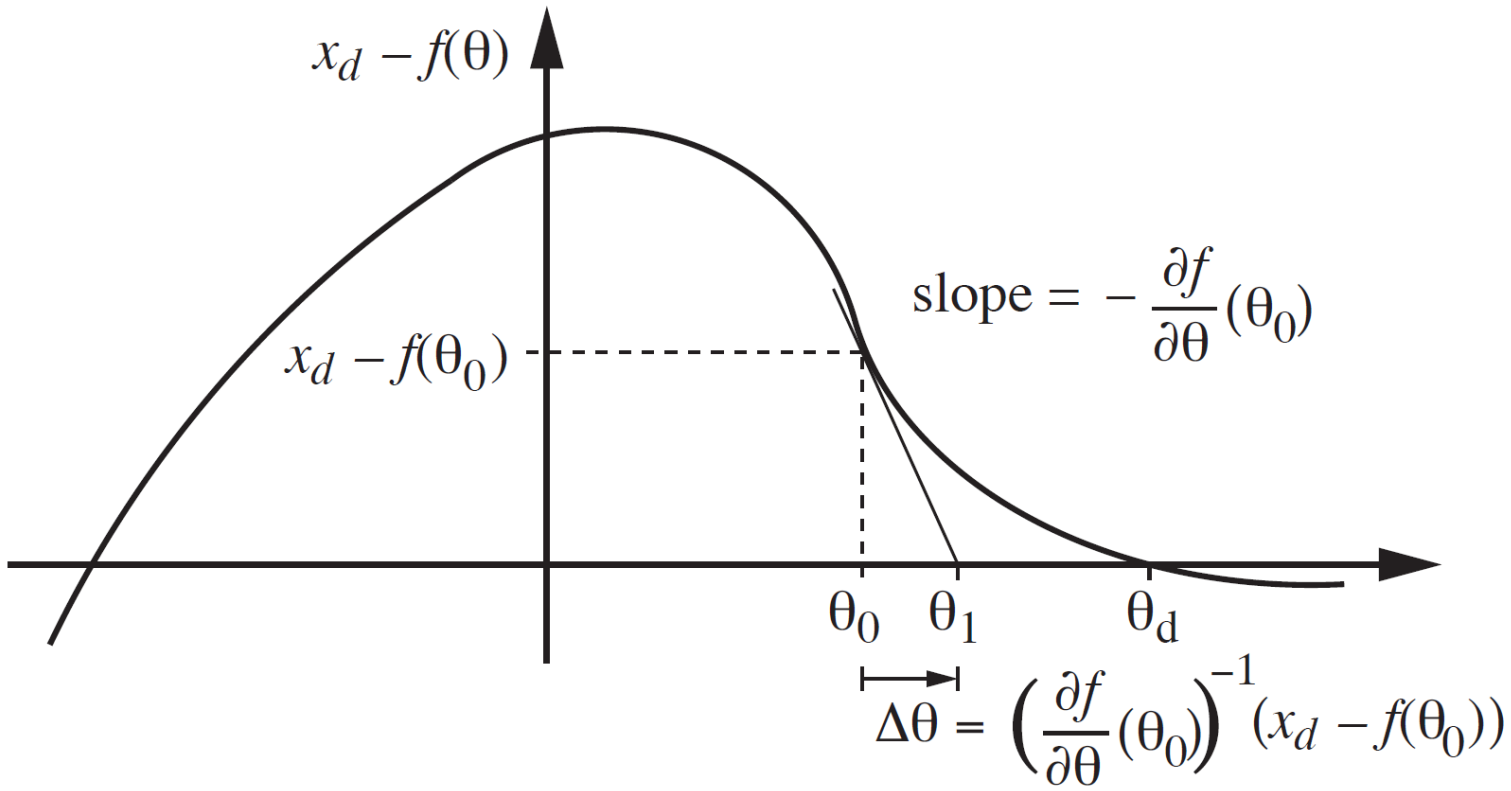
end while

Convergence to Different Solutions

Newton-Rhapson is a **locally optimal** method.

It will **converge** to the solution **closest to the initial guess**.

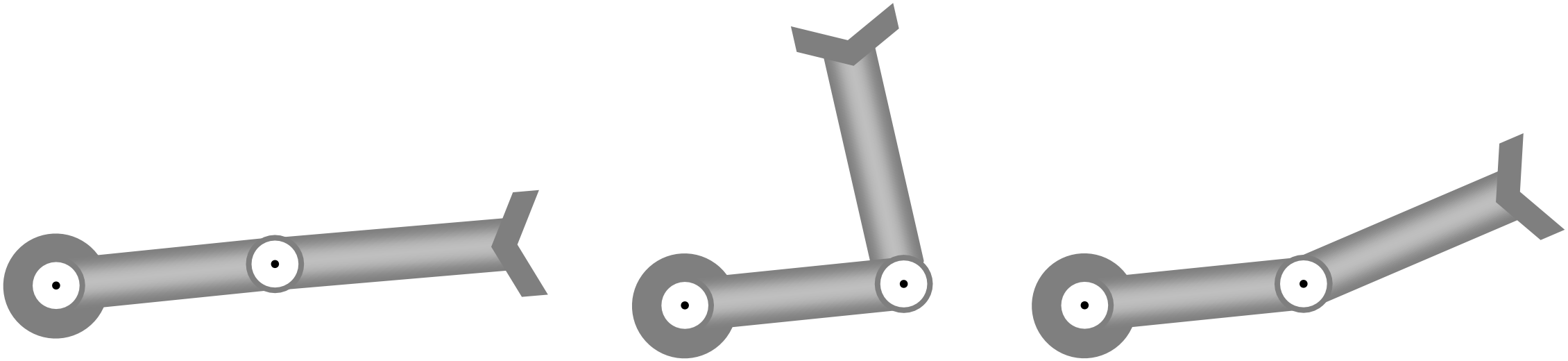
It is therefore **highly-dependent on the initial guess**.



Part II: Manipulability

Let us start with a question…

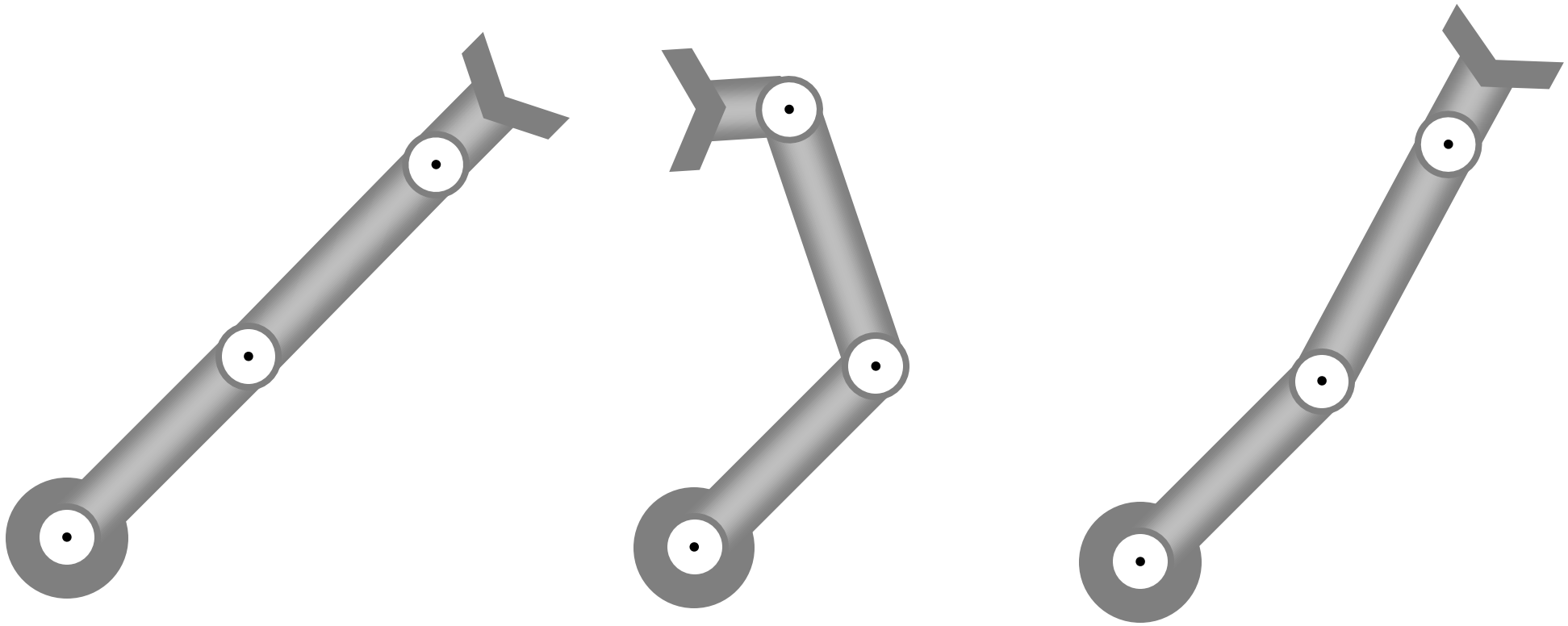
What can you say about these three robot configurations?:



Can you justify this mathematically?
(in two different ways)

Let us start with a question…

What if we instead have this robot?



Can you justify this mathematically in the same two ways?

What do we want to arrive at?

Remember that **the structure of the Jacobian** can tell use **whether we are at a singularity**.

If we are at a **singularity:**

- The Jacobian matrix **loses rank.**
- Its **determinant becomes zero** → If we are **close to a singularity**, the **determinant approaches zero**.

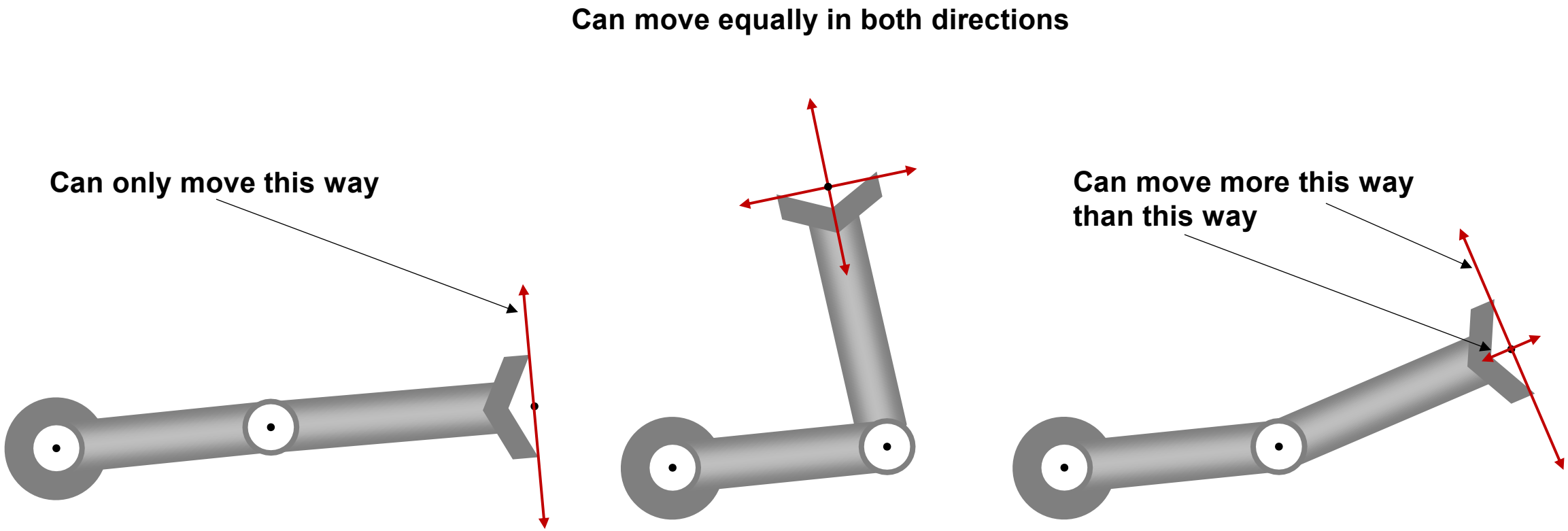
But, what if the robot is redundant?

- The **Jacobian is NOT square** → The **determinant is not defined**.

And what if we want to say **which direction** (in Cartesian space) is **closest to a singularity**?

…In other words (in other pictures?)

We want to derive something that can tell us this:



Manipulability Measure

At a very **basic level**:

For a **non-redundant** robot:

- We can see how close we are to a singularity by looking at the **determinant of the Jacobian**:

$$\mu = \det(\boldsymbol{J}) \qquad \begin{array}{ll} \mu \approx 0 & \text{close} \\ \mu > 0 & \text{not close} \end{array}$$

For a **redundant** robot:

- The determinant does not exist.
- We use a typical math cheat:

$$\mu = \sqrt{\det(\boldsymbol{J}\boldsymbol{J}^T)} \qquad \begin{array}{ll} \mu \approx 0 & \text{close} \\ \mu > 0 & \text{not close} \end{array}$$

But we would like to say more than just ”close”/”not close”.

As usual, before we get there, we need to review some math…

Let us look at something in MATLAB…

…can you explain what is happening?

Matrices as Linear Transformations

A **matrix** can be interpretted as a **geometric operator on a vector**, particularly as a **linear transformation**.

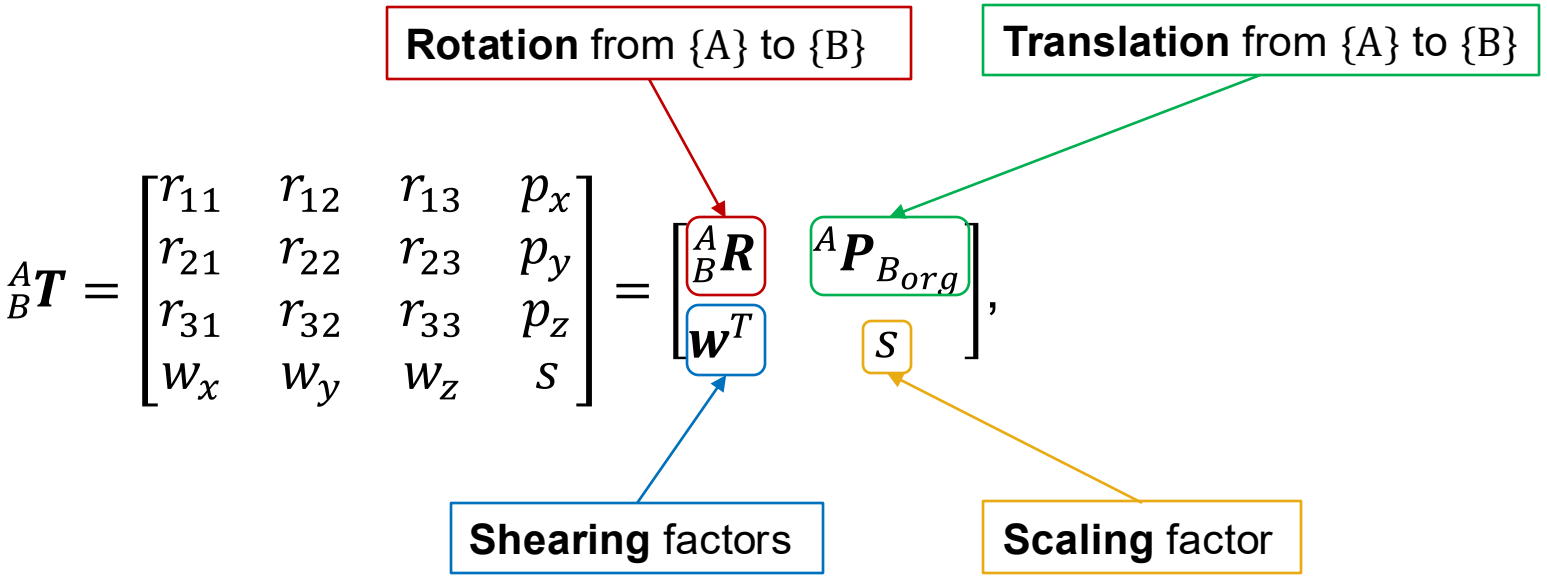
If we have: $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$, where $\boldsymbol{x}$ and $\boldsymbol{y}$ are vectors and $\boldsymbol{A}$ is a matrix, $\boldsymbol{A}$ can either:

- **Scale**
- **Rotate**
- **Shear**
- **A combination of the above**

the vector $\boldsymbol{x}$, to produce a new scaled/rotated/sheared vector $\boldsymbol{y}$.

Homogenous Transformations Matrices

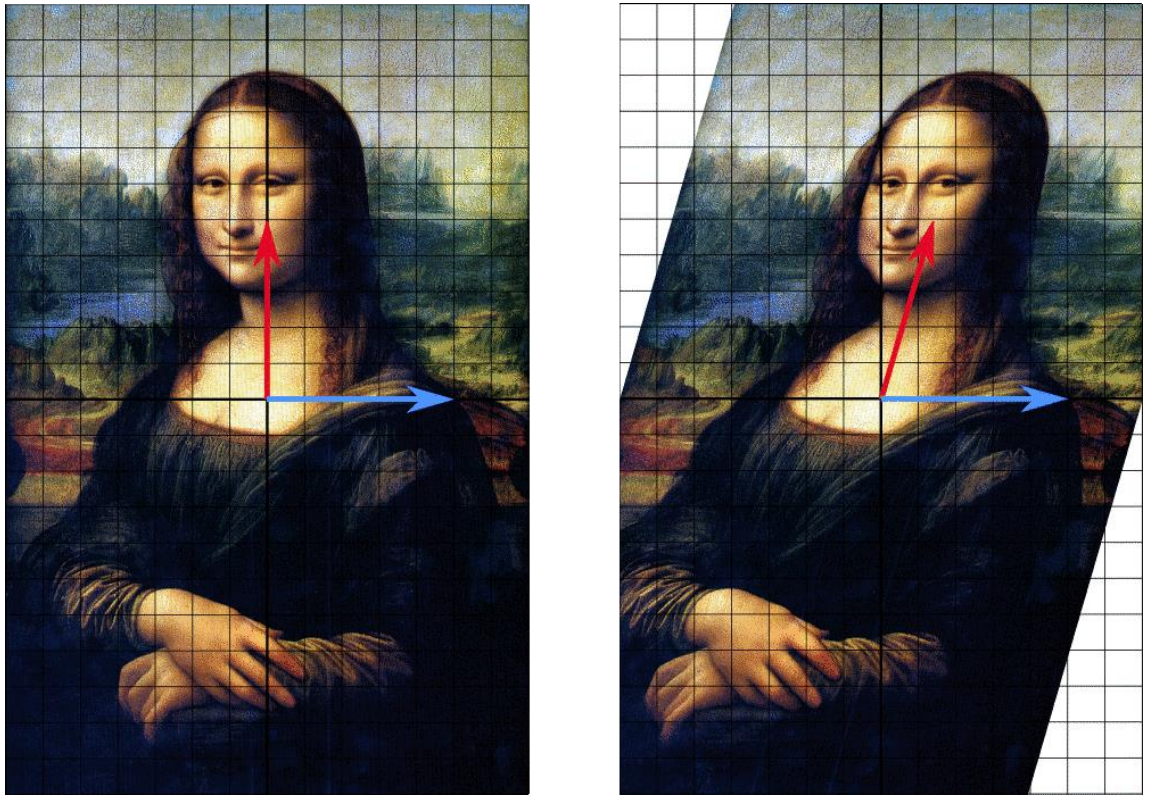
All of this **should NOT be a huge suprise**… Remember homogenous **transformation matrices**:



Expanding the Concept to ALL vectors

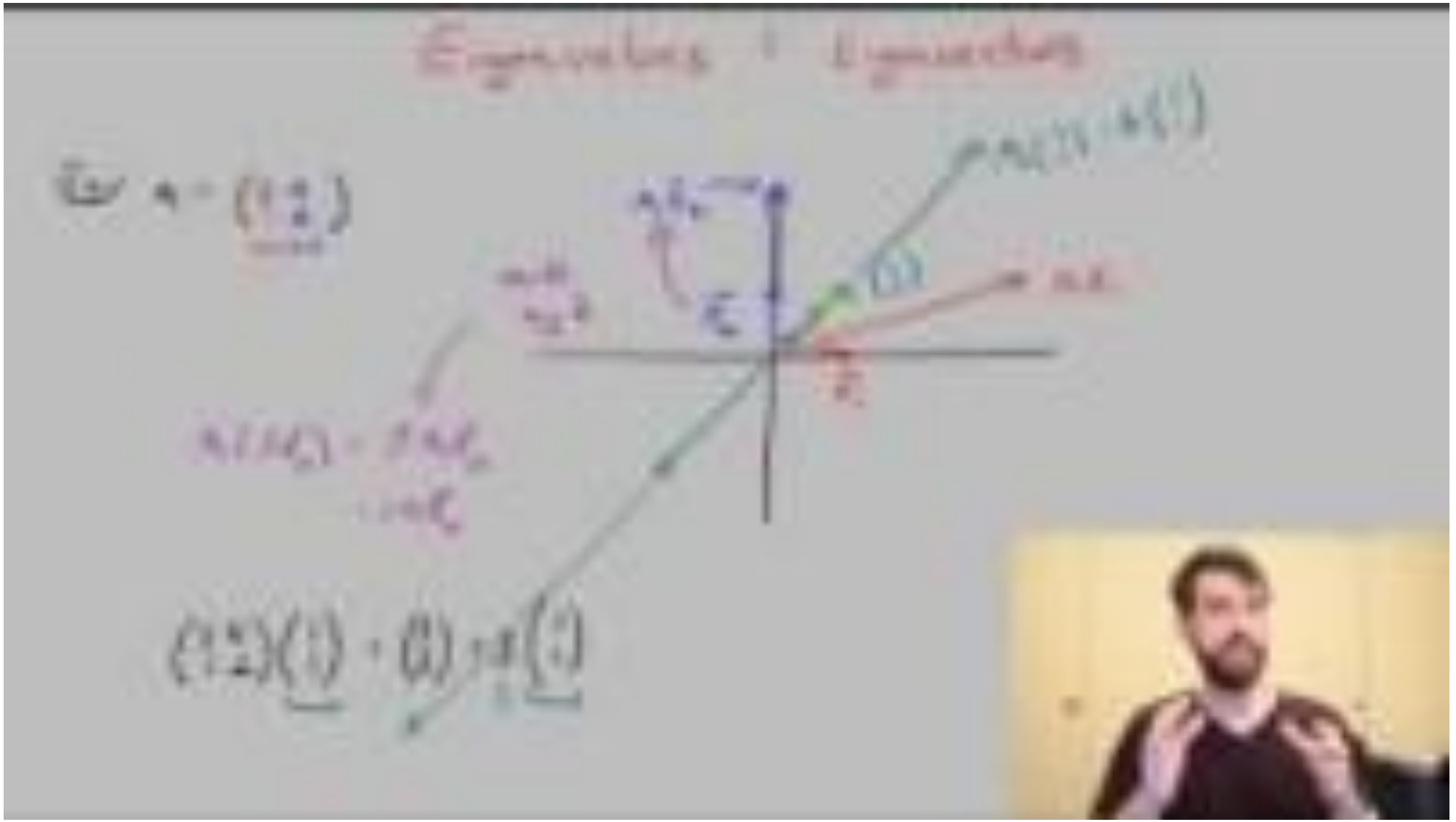
Now think of the **same concept**, but applied to **every possible vector in that space**.

For example, see it as a **transformation on an image** (e.g. shearing):



SDU ROBOTICS

Eigen-vectors and Eigen-values: $Ax = \lambda x$



SDU ROBOTICS

Back to our Robots…

From forward velocity kinematics, we have:

$$\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$$

If we write: $\dot{\boldsymbol{q}}^T\dot{\boldsymbol{q}} = 1$

We can substitute: $\left(\boldsymbol{J}^{-1}\dot{\boldsymbol{x}}\right)^T\left(\boldsymbol{J}^{-1}\dot{\boldsymbol{x}}\right) = 1$

Re-arrange:

$$\dot{\boldsymbol{x}}^T\boldsymbol{J}^{-T}\boldsymbol{J}^{-1}\dot{\boldsymbol{x}} = 1$$
$$\dot{\boldsymbol{x}}^T(\boldsymbol{J}\boldsymbol{J}^T)^{-1}\dot{\boldsymbol{x}} = 1$$

And define:

$$\boldsymbol{A} = \boldsymbol{J}\boldsymbol{J}^T$$

Such that:

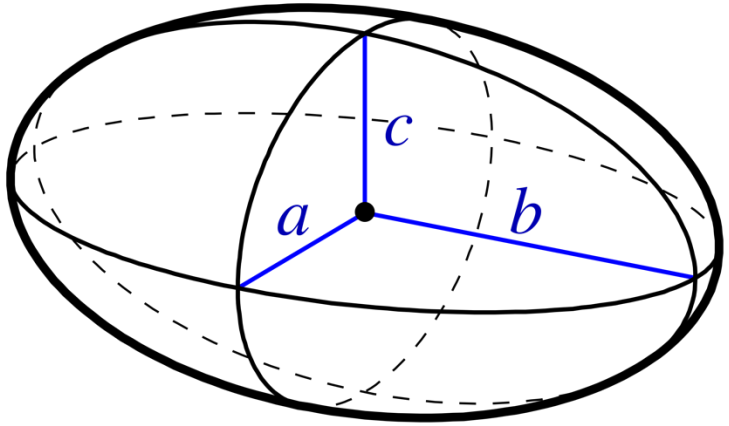
$$\dot{\boldsymbol{x}}^T\boldsymbol{A}^{-1}\dot{\boldsymbol{x}} = 1$$

Deriving an Ellipsoid

From the definitions in the previous slide, we have:

$$\boldsymbol{A} = \boldsymbol{J}\boldsymbol{J}^T$$

$$\dot{\boldsymbol{x}}^T \boldsymbol{A}^{-1} \dot{\boldsymbol{x}} = 1$$



Assuming $\boldsymbol{A}$ is positive semi-definite, **this** is the **matrix equation for an ellipsoid**.

There is a proof that states:

For any matrix, $\boldsymbol{M}$:

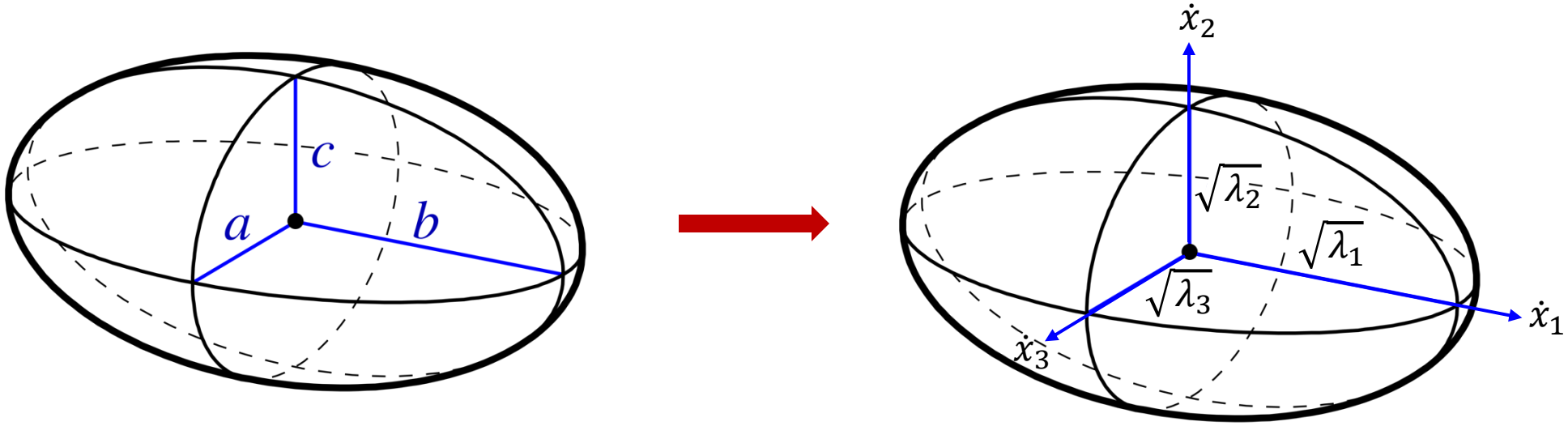
$\boldsymbol{M}\boldsymbol{M}^T \geq 0$ **Symmetric, positive semi-definite**

Good, we have an ellipsoid.

Manipulability Ellipsoid

In the equation: $\dot{\boldsymbol{x}}^T \boldsymbol{A}^{-1} \dot{\boldsymbol{x}} = 1$

The **axes $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$ of the ellipsoid** can be **related to the eigen-values** and **eigen-vectors** of $\boldsymbol{A}$.



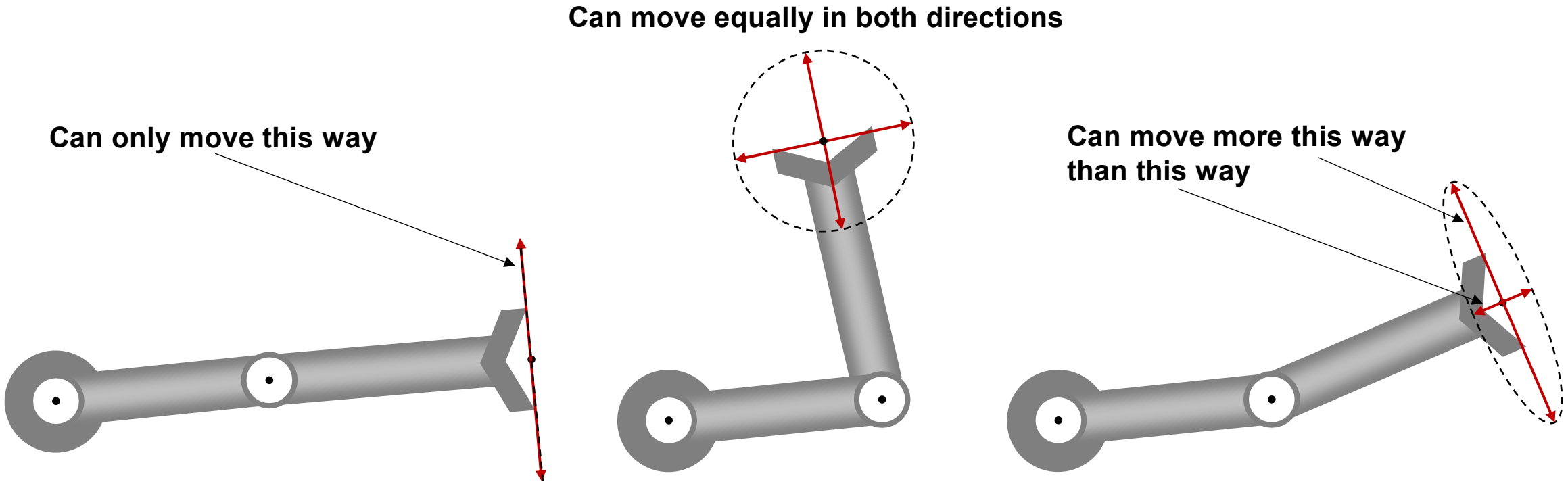
Note: Usually only the positional part of the Jacobian, $\boldsymbol{J_P}$, is used to calculate the manipulability ellipsoid.

Putting it all together

In a 2D example our ellipsoid becomes an ellipse.

It tells us exactly what we wanted!





More Manipulability Measures

Measure #1 *(same as before)*:

$$\mu = \sqrt{\det(\boldsymbol{J}\boldsymbol{J}^T)} = \det(\boldsymbol{A})$$

$\mu \approx 0$ close
$\mu > 0$ not close

Measure #2:

$$\boldsymbol{A} = \boldsymbol{J}\boldsymbol{J}^T$$

$$\mu = \frac{\sqrt{\lambda_{\max}(\boldsymbol{A})}}{\sqrt{\lambda_{\min}(\boldsymbol{A})}} \geq 1$$

μ is large → close to singularity
μ is small → far from singularity

Measure #3:

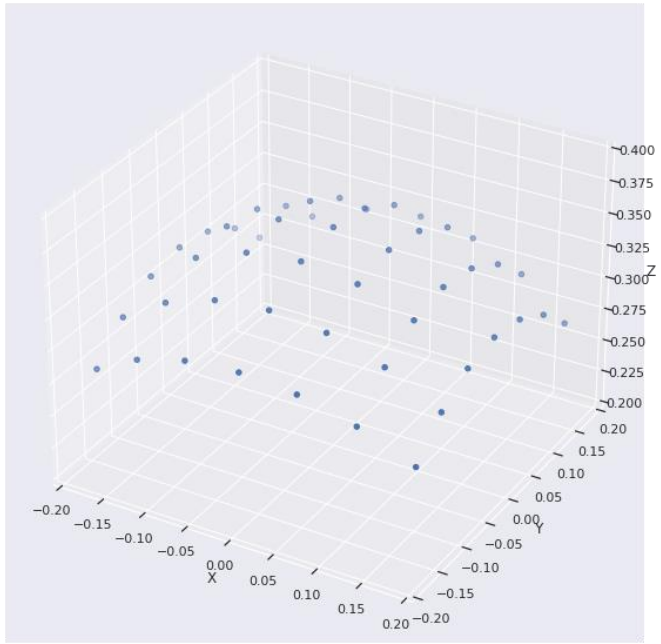
$$\mu = \frac{\lambda_{\max}(\boldsymbol{A})}{\lambda_{\min}(\boldsymbol{A})} \geq 1$$

μ is large → close to singularity
μ is small → far from singularity

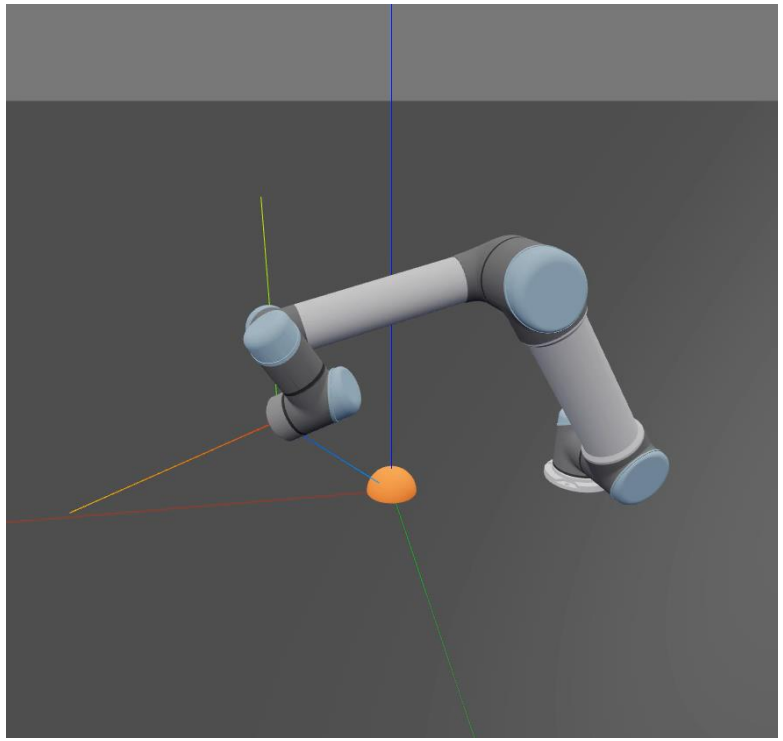
Manipulability: Practical Application

Manipulability is **very useful**, for example to **choose where to place a robot** in a workcell:

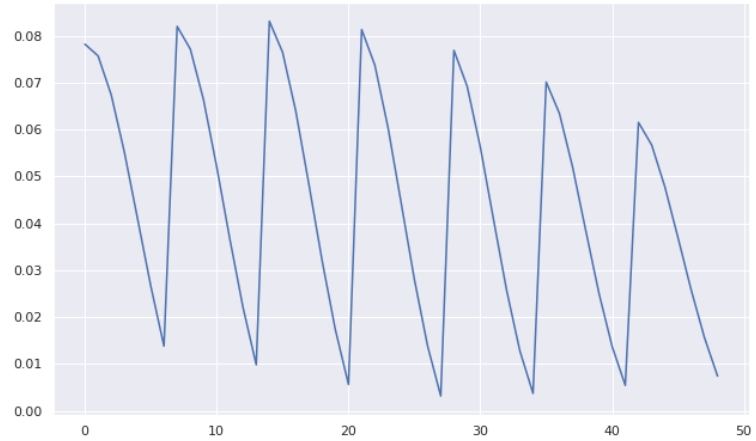
1. Make a grid along the workspace.



2. Move the robot along the grid.



3. For each point, calculate the manipulability.



Let us see it in MATLAB…

Recap: What have we discussed today

- The Newton-Raphson algorithm for **numerical inverse kinematics**:
  1. Uses the **Jacobian** matrix to **convert between joint and end-effector velocities**.
  2. Relies on making a joint-space movement approximately linear by **taking very small steps**.
  3. Converges to a **locally optimal solution** (that depends on the start configuration).

- **Manipulability** gives us a measure of **how close the robot is to a singularity**.
  - **Manipulability metrics** are based on the **determinant of the Jacobian matrix.**
  - By performing **eigendecomposition of the manipulability** metric, we can draw a **manipulability ellipsoid**
  - The **manipulability ellipsoid** gives us information on **which directions of movement** of the end-effector move closer to singularities.

Take home message(s):

The **Jacobian** can be used (among other things):

To calculate **numerical inverse kinematics** using the Newton-Rhapson algorithm.

To determine **proximity to singularities** using **manipulability metrics**.

SDU ROBOTICS